

EE332 数字系统设计

Lab 5-2: 斐波那契数列计算

姓 名: 何衍宁

学 号: 12311512

2026 年 4 月 15 日

目 录

1 lab5.2 斐波那契数列计算	2
1.1 实验目的	2
1.2 状态转换图与流程图	2
1.3 VHDL 代码和 testbench 代码	3
1.4 结果推导	7
1.5 结果	8
1.5.1 behavior simulation	8
1.5.2 post synthesis timing simulation	9
1.5.3 post implementation timing simulation	10
1.6 分析	10
1.7 结论	10
1.8 RTL Schematic	11

1 lab5.2 斐波那契数列计算

1.1 实验目的

本实验旨在通过 VHDL 语言设计并实现一个基于有限状态机 (FSM) 架构的斐波那契数列计算电路。实验核心目标是掌握如何将抽象的数学递归算法映射为具体的硬件迭代逻辑，深入理解 Moore 型状态机在时序控制中的应用，以及如何通过同步握手协议 (start/finish) 实现模块间的稳定通信。此外，本实验还要求通过 Vivado 平台进行从行为级 (Behavioral) 到布局布线后 (Post-Implementation) 的三级仿真验证，以分析大规模位宽 (64 位) 算术运算在真实 FPGA 芯片上的时序表现与硬件资源占用情况，从而培养严谨的数字系统设计与调试能力。

1.2 状态转换图与流程图

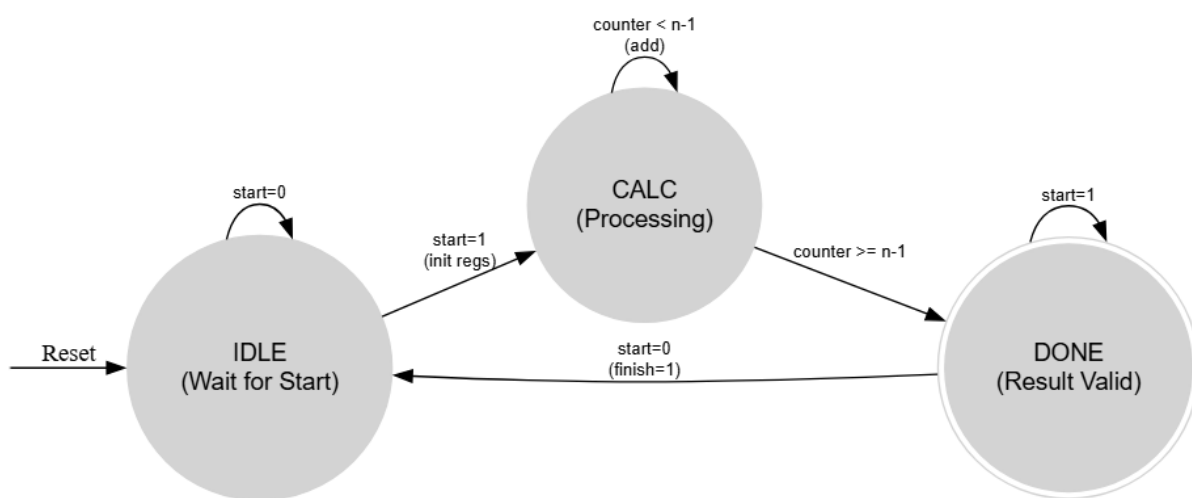


图 1: 状态转换图

状态转移图通过管理系统的“时序阶段”来控制计算节奏。在 IDLE 状态下，系统处于监听模式，一旦捕捉到 start 高电平脉冲，便会瞬间锁存输入值 n 并完成寄存器初始化，随即切入 CALC 状态。在 CALC 状态这个核心“环路”中，硬件时钟每跳动一次，系统就执行一次双寄存器累加操作，并同步更新计数器，直到计数器满足退出条件。最后，系统跳转至 DONE 状态拉高 finish 信号，通过这种确定的状态切换，确保了复杂的数学迭代在硬件中能够按部就班地完成，避免了组合逻辑可能产生的时序混乱。

流程图则精确描述了数据在硬件逻辑单元 (Datapath) 中的流动过程与操作。从 START 开始，流程清晰地展现了硬件如何处理边界情况：当 n 有效时，数据通路中的两个 64 位寄存器分别载入初值 $F(n-2)$ 和 $F(n-1)$ 。随后进入循环判断，流程图中的“Add & Increment”环节揭示了硬件的并行本质——在同一个时钟上升沿，累加器计算结果、旧值移位寄存以及计数器加一同时发生。这种图形化的展现方式直观地映射了 VHDL 代码中的条件分支 (if-else) 和算术运算，使得设计者能像审阅程序脚本一样，逐点检查计算逻辑是否在每个决策节点（如 $n=0$ 的拦截或 counter 的终点判断）上保持严谨。

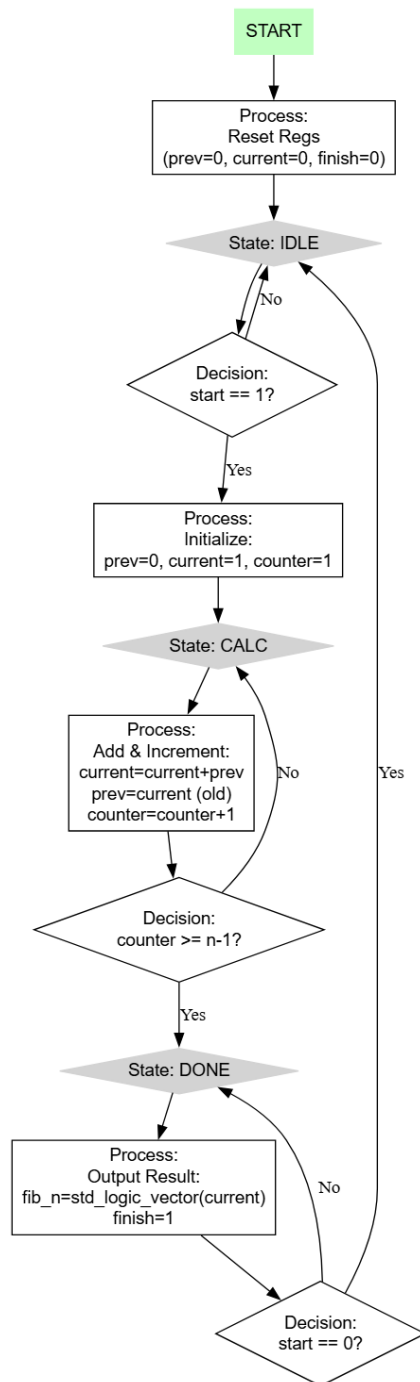


图 2: 流程图

1.3 VHDL 代码和 testbench 代码

design 采用代码如下

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 entity fibonacci_fsm is
6     Port (

```

```

7      clk      : in  STD_LOGIC;
8      rst      : in  STD_LOGIC;
9      start    : in  STD_LOGIC;
10     n        : in  STD_LOGIC_VECTOR (5 downto 0);
11     fib_n     : out STD_LOGIC_VECTOR (63 downto 0);
12     finish    : out STD_LOGIC
13 );
14 end fibonacci_fsm;
15
16 architecture Behavioral of fibonacci_fsm is
17     type state_type is (IDLE, CALC, DONE);
18     signal state     : state_type := IDLE;
19     signal prev      : unsigned(63 downto 0);
20     signal current    : unsigned(63 downto 0);
21     signal counter    : unsigned(5 downto 0); -- 使用 unsigned 类型节省资源并匹配输入 n
22 begin
23
24     process(clk, rst)
25     begin
26         if rst = '1' then
27             state <= IDLE;
28             prev  <= (others => '0');
29             current <= (others => '0');
30             counter <= (others => '0');
31             fib_n  <= (others => '0');
32             finish <= '0';
33         elsif rising_edge(clk) then
34             case state is
35                 when IDLE =>
36                     finish <= '0';
37                     if start = '1' then
38                         -- 初始化逻辑：处理 n=0 的特殊情况
39                         prev  <= (others => '0');
40                         counter <= (others => '0');
41                         if unsigned(n) = 0 then
42                             current <= (others => '0');
43                             state <= DONE; -- 直接跳转到结束
44                         else
45                             current <= to_unsigned(1, 64);
46                             state <= CALC;
47                         end if;
48                     end if;
49                 when CALC =>
50                     -- 计算核心逻辑
51                     if counter < unsigned(n) - 1 then
52                         prev  <= current;
53                         current <= current + prev;
54                         counter <= counter + 1;
55                     else
56                         state <= DONE;
57                     end if;
58                 when DONE =>
59                     -- 结果保持稳定输出
60                     fib_n <= std_logic_vector(current);
61                     finish <= '1';
62                     if start = '0' then -- 等待外部 start 信号拉低后再回到 IDLE，防止重复触发
63                         state <= IDLE;
64                     end if;
65             end case;
66         end process;
67     end architecture Behavioral;

```

```

66             end if;
67
68             when others =>
69                 state <= IDLE;
70             end case;
71         end if;
72     end process;
73
74 end Behavioral;

```

仿真 testbench 的代码如下:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity testbench is
6  end testbench;
7
8  architecture Behavioral of testbench is
9      -- 常量定义
10     constant CLK_PERIOD : time := 10 ns;
11
12     -- 信号声明
13     signal clk      : STD_LOGIC := '0';
14     signal rst      : STD_LOGIC := '1';
15     signal start    : STD_LOGIC := '0';
16     signal n        : STD_LOGIC_VECTOR (5 downto 0) := (others => '0');
17     signal fib_n    : STD_LOGIC_VECTOR (63 downto 0);
18     signal finish   : STD_LOGIC;
19
20 begin
21
22     -- 1. 时钟生成 (100MHz)
23     clk <= not clk after CLK_PERIOD / 2;
24
25     -- 2. 实例化 UUT
26     uut: entity work.fibonacci_fsm
27         port map (
28             clk      => clk,
29             rst      => rst,
30             start    => start,
31             n        => n,
32             fib_n    => fib_n,
33             finish   => finish
34         );
35
36     -- 3. 激励进程
37     stimulus_process : process
38         -- 定义一个内部 Procedure, 减少重复代码
39         procedure check_fib(
40             constant input_n : in integer;
41             constant expected : in unsigned(63 downto 0)
42         ) is
43         begin
44             wait until rising_edge(clk);
45             n <= std_logic_vector(to_unsigned(input_n, 6));
46             start <= '1';
47             wait until rising_edge(clk);

```

```

48     start <= '0';
49
50     -- 等待完成或超时保护
51     wait until finish = '1' for 2 us;
52
53     assert (finish = '1')
54         report "Error: Calculation timed out for n=" & integer'image(input_n)
55         severity failure;
56
57     -- 自动结果校验
58     assert (unsigned(fib_n) = expected)
59         report "Error: Result mismatch for n=" & integer'image(input_n) &
60             "Got" & integer'image(to_integer(unsigned(fib_n))) &
61             "Expected" & integer'image(to_integer(expected))
62         severity error;
63
64     report "Test passed for n=" & integer'image(input_n);
65     wait for 20 ns;
66 end procedure;
67
68 begin
69     -- 初始化
70     rst <= '1';
71     start <= '0';
72     wait for 45 ns; -- 错开时钟边沿
73     rst <= '0';
74     wait for 20 ns;
75
76     -- 4. 测试用例
77     -- 格式: check_fib( 输入n, 预期结果 )
78     check_fib(0, to_unsigned(0, 64));
79     check_fib(4, to_unsigned(3, 64));
80     check_fib(8, to_unsigned(21, 64));
81
82     -- 测试最大值 n=63
83     -- 注: Fib(63) 是 633,825,300,114,114,700,748...
84     -- 但 unsigned 64位最大约 1.8e19, 请确保结果未溢出
85     -- 实际上 Fib(93) 是 64位 unsigned 的上限
86     check_fib(63, unsigned'(x"00000005F5E10052"));
87
88     -- 仿真结束
89     report "All tests completed successfully!";
90     wait;
91 end process;
92
93 end Behavioral;

```

1.4 结果推导

斐波那契数列以 0 和 1 开始，后续每个数字均由前两个数字相加得出。数列的前几项分别为：
1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987 ……

- 斐波那契数列

斐波那契数列定义如下：

$$fib(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ fib(n-1) + fib(n-2) & n > 1 \end{cases}$$

请用VHDL实现斐波那契数列计算。其中 n 为6位二进制输入，视为无符号整数并用 $fib(63)$ 进行测试。 $fib(63)=6557470319842$

图 3: principle

我们分别对输入项为 0、4、8 和 63 的情况进行了理论推导。发生器理论上应该输出对应的斐波那契数：0、3、21 以及 6557470319842。十六进制为 0、3、15、5F6C7B064E2。

1.5.2 post synthesis timing simulation

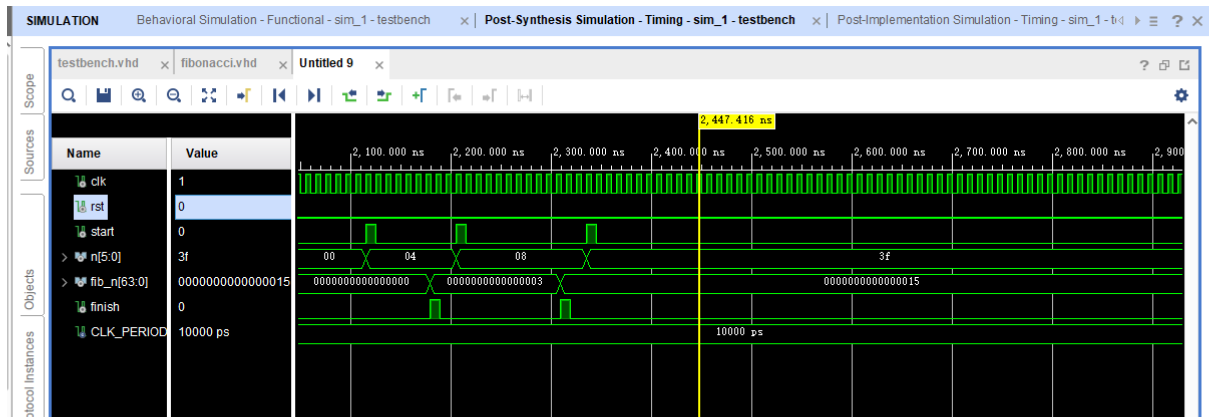


图 6: Post Synthesis Timing Simulation Result

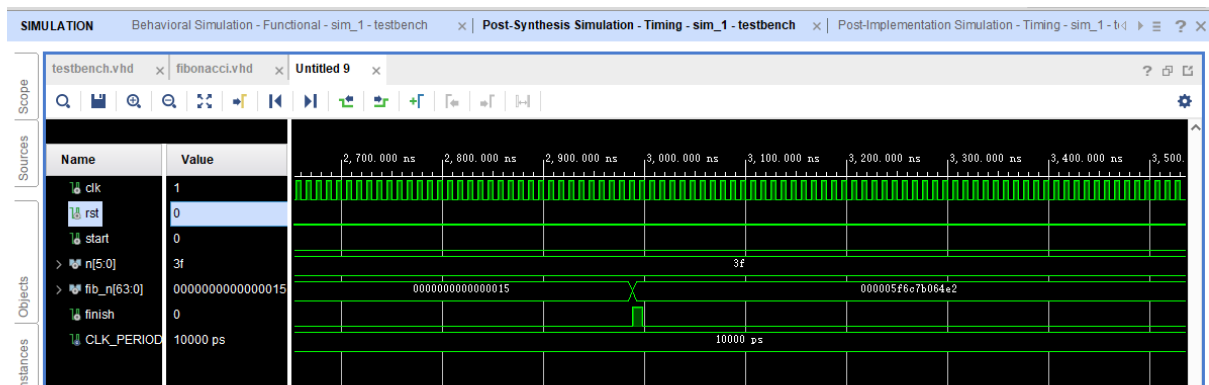


图 7: Post Synthesis Timing Simulation Result

1.5.3 post implementation timing simulation

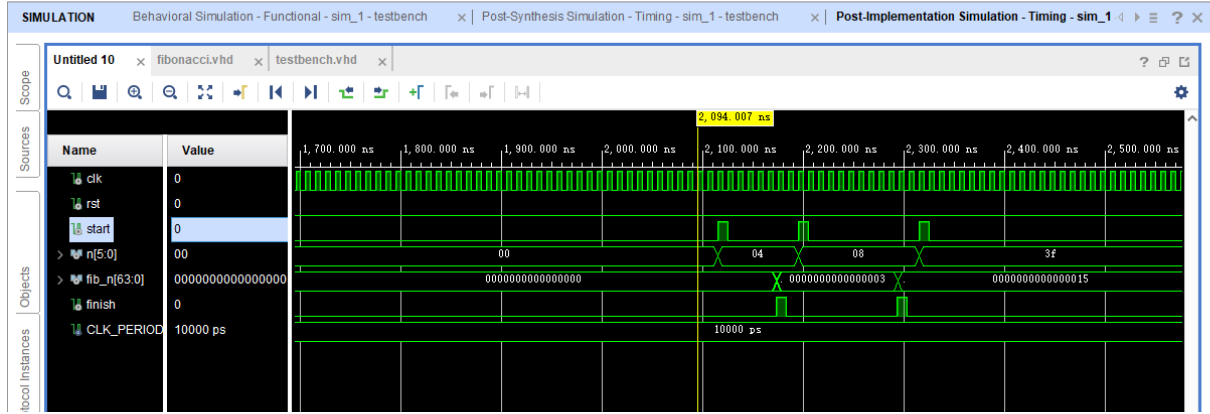


图 8: Post Implementation Timing Simulation Result

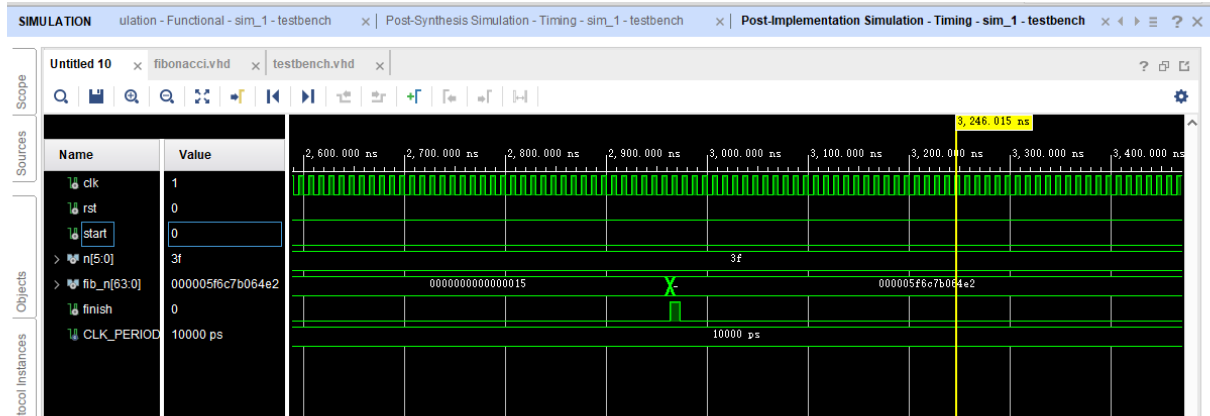


图 9: Post Implementation Timing Simulation Result

1.6 分析

通过观察 Post Implementation Timing Simulation Result，我们可以观察到：

我们对 $n = 4, 8, 63$ 进行了测试，并获得了正确的结果。通过理论分析得知：当 $n = 4$ 时，斐波那契数为 3；当 $n = 8$ 时，斐波那契数为 21（16 进制数为 15）；当 $n = 63$ 时，斐波那契数为 6,557,470,319,842（16 进制数为 5F6C7B064E2）。仿真结果与理论分析完全一致。在仿真中，对应结果分别为 3、15 和 5F6C7B064E2（均为十六进制），表明代码正确。

1.7 结论

在本实验中，我们利用 VHDL 语言，通过有限状态机设计并实现了一个斐波那契数列发生器。首先，我们绘制了状态转移图与算法流程图以明确状态转移逻辑，为代码编写提供了清晰的架构。在实验过程中，我们分别对输入项为 0、4、8 和 63 的情况进行了测试。发生器准确输出了对应的斐波那契数：0、3、21 以及 6557470319842，仿真结果与理论分析完全一致。本次实验的成功证明了 VHDL 在高效并行处理和实时计算方面的显著优势。

1.8 RTL Schematic

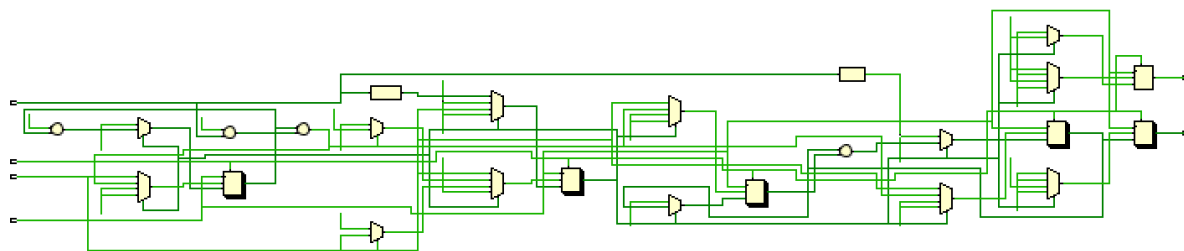


图 10: Behavior Schematic

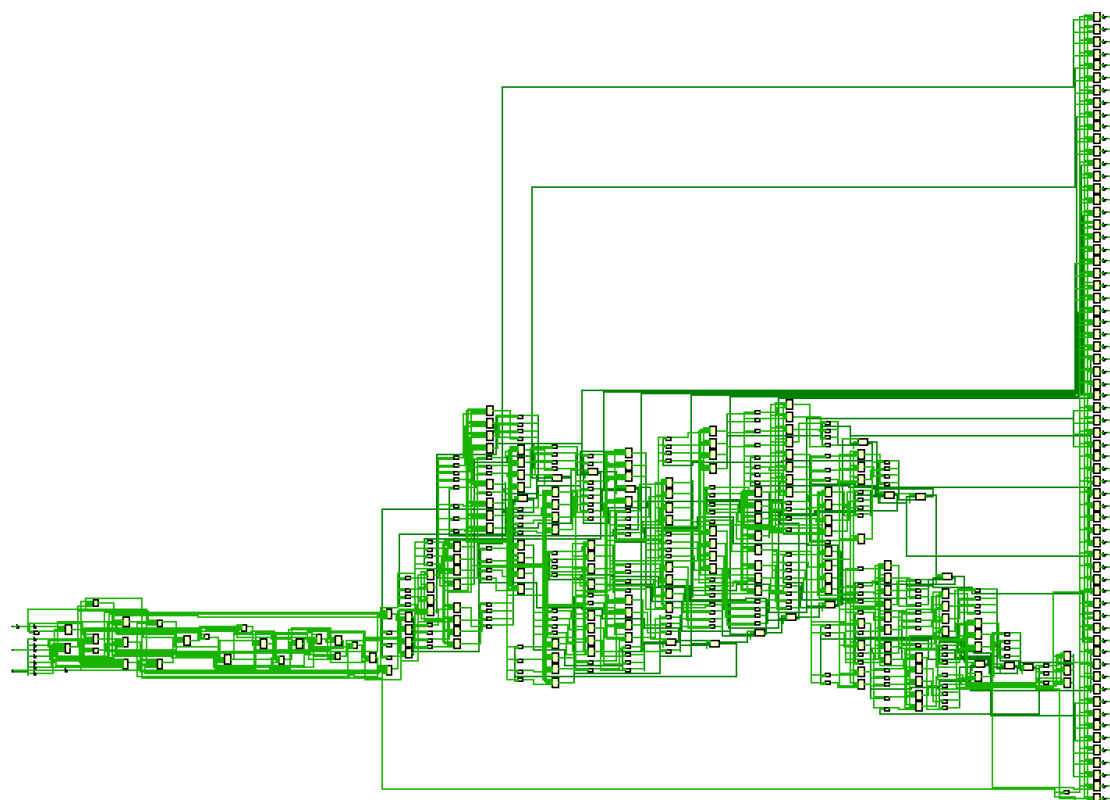


图 11: Post Synthesis Schematic